

# Stereo



Slide credits: Ioannis Gkioulekas, Kris Kitani, Srinivasa Narasimhan

## Overview of today's lecture

- Revisiting triangulation.
- Disparity.
- Stereo rectification.
- Stereo matching.
- Improving stereo matching.
- Structured light.

# Structure from motion



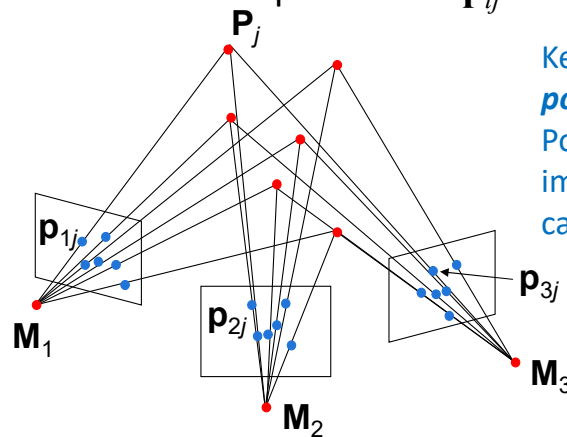
Драконъ, видимый подъ различными углами зрѣнія  
По гравюру на деревѣ изъ „Oculus artificialis teleiopicus“ Иона. 1702 года.

# Structure from motion

- Given:  $m$  images of  $n$  fixed 3D points

$$\bullet \mathbf{p}_{ij} = \mathbf{M}_i \mathbf{P}_j, \quad i = 1, \dots, m, \quad j = 1, \dots, n$$

- Problem: estimate  $m$  projection matrices  $\mathbf{M}_i$  and  $n$  3D points  $\mathbf{P}_j$  from the  $mn$  correspondences  $\mathbf{p}_{ij}$



Key assumption: **matching points**  
Possible only for textured image areas where features can be detected!

## Sparse point cloud reconstruction



## Revisiting triangulation

# How would you reconstruct 3D points?



Left image



Right image

Let's say that we have two images of the same scene, captured with two calibrated cameras. We assume that we have computed the fundamental matrix relating the two cameras, either using their calibrated intrinsics and extrinsics, or using the 8-point algorithm.

## How would you reconstruct 3D points?



Left image



Right image

1. Select point in one image (how?)

We can then run a feature point detector, such as the Harris corner detector we covered in the Feature Detection and Matching lecture, to select a few important points on the left image.



## How would you reconstruct 3D points?



Left image



Right image

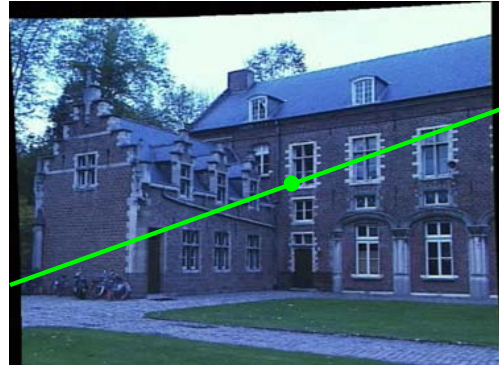
1. Select point in one image (how?)
2. Form epipolar line for that point in second image (how?)

For each selected point, we use the fundamental matrix to form the corresponding epipolar line in the second image.

## How would you reconstruct 3D points?



Left image



Right image

1. Select point in one image (how?)
2. Form epipolar line for that point in second image (how?)
3. Find matching point along line (how?)

We then search this epipolar line to find the matching point. For this, we can use the SIFT feature descriptor, computed at all points along the line. Or, we can use simpler patch matching procedures, and just compare the sum of square differences over the patch.

## How would you reconstruct 3D points?



Left image

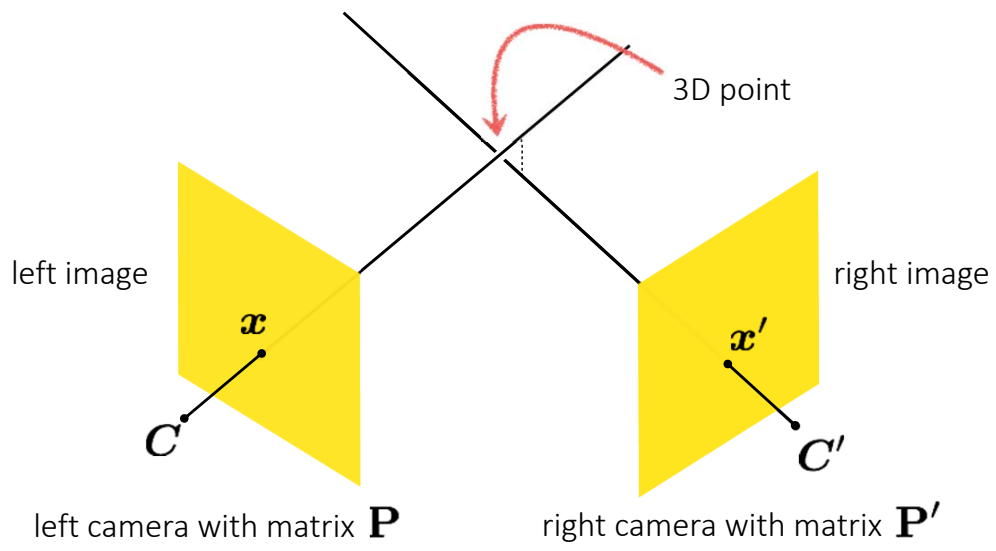


Right image

1. Select point in one image (how?)
2. Form epipolar line for that point in second image (how?)
3. Find matching point along line (how?)
4. Perform triangulation (how?)

Lastly, we can perform triangulation on the two matched points, to compute the corresponding 3D point in the scene. This gives us a full pipeline for doing 3D reconstruction!

## Triangulation



## How would you reconstruct 3D points?



Left image



Right image

1. Select point in one image (how?)
2. Form epipolar line for that point in second image (how?)
3. Find matching point along line (how?)
4. Perform triangulation (how?)

However, this pipeline is not without issues: The main problem here is that, for each 3D point we reconstruct, we need to solve a linear system for triangulation. If our image has size 1 megapixel, this means that we need to solve a million linear systems, which quickly becomes computationally intractable. Because of this, the pipeline we just described is typically used to do 3D reconstruction only on a sparse set of pixels selected by a feature detector, such as the Harris corner detector.

Today we will see a different pipeline for 3D reconstruction that can be used to produce dense depth maps for all pixels in an image.

## Discrete vs. continuous depth

- Depth reconstruction requires a pair of matching points  $p, p'$  as input. This matching can only be applied at image regions with considerable texture (e.g. corners) rather than uniform regions
- Next discuss the concept of stereo matching, attempting to interpolate depth values in texture-less regions.

Disparity



*What's different between these two images?*

Today we will derive a different procedure for performing 3D reconstruction without triangulation. Our procedure will be based on the notion of disparity. Before we dive into the math, in the coming slides we will provide some intuition about what disparity is and how it relates to depth. For this, consider these two images of the same scene, captured by a camera performing a horizontal motion from left to right. To better see how the images differ, it helps to flip between the two.





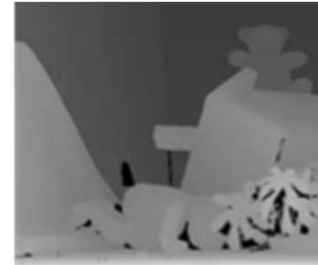




*Objects that are close move more or less?*

What we observe is that, as the camera moves horizontally from left to right, all objects appear to move horizontally in the image frame. However, the amount by which they move differs, depending on the distance of the objects from the camera. Objects that are closer to the camera move by a bigger amount between the two views, than objects that are further from the camera.

*The amount of horizontal movement is  
inversely proportional to ...*



*Disparity image. Brightness in this image  $d(p)$ , encodes the amount of shift between the images at pixel  $p$ .*

*... the distance from the camera.*

We call the amount of horizontal movement for each pixel the *disparity*. From the observation we discussed earlier, the disparity of a pixel appears to be inversely related to the distance of the object shown in that pixel from the camera; that is, disparity is inversely related to the depth of the object. In the next slides, we will see exactly how to relate disparity to depth, and how to use disparity to compute dense depth maps for a scene, such as what is shown here. We will refer to this procedure as depth from disparity, or stereo.

### *Real-time stereo sensing*

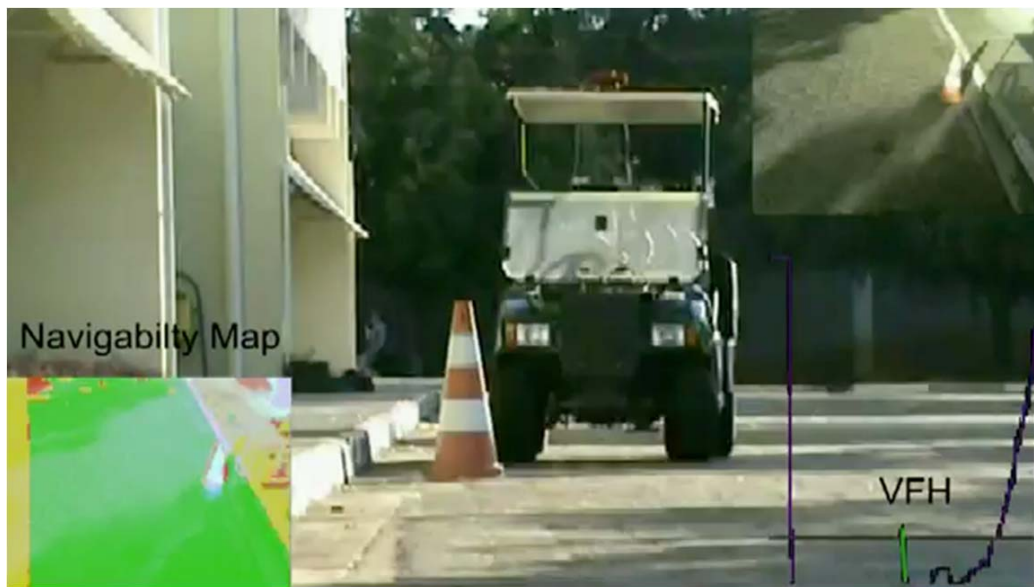


*Nomad robot searches for meteorites in Antarctica*

<http://www.frc.ri.cmu.edu/projects/meteorobot/index.html>

Stereo is one of the most widely employed, if not *the* most widely employed, 3D sensing algorithm. It is used across all sorts of AI systems, from robots for autonomous exploration...

To modern vehicles for driving assistance and autonomous driving. In fact, nowadays even a lot of cell phones have at least two cameras, which makes it possible for them to use stereo for 3D sensing.





*Subaru  
Eyesight system*

*Pre-collision  
braking*







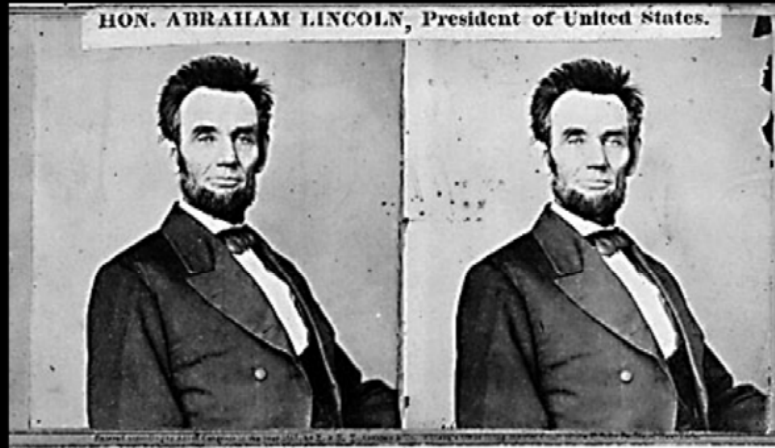
## *What other vision system uses disparity for depth sensing?*

In addition to robots, cars, and cell phones, stereo is one of the main 3D sensing mechanisms used by the human vision system, with our two eyes acting as the two cameras. This is the reason why, by manipulating the way we show 2D images to our two eyes, we can fool our brain into thinking we are looking at a real 3D scene.

## Stereoscopes: A 19<sup>th</sup> Century Pastime



People had realized this in the 19th century, creating devices such as the one shown here called stereoscopes.



These forced each eye to only look at one of two photographs, which depicted the same scene from *slightly* shifted viewpoints. Due to the disparity between the two images, the brain thinks the viewer is looking at a real 3D scene of whatever is shown in the pair of images; in this example, President Abraham Lincoln.



Public Library, Stereoscopic Looking Room, Chicago, by Phillips, 1923



The same effect can be achieved using colored glasses, rather than a stereoscope. We can form an image, whose red and blue channels correspond to images of the same scene captured under slightly shifted views. Using colored glasses, the left eye will see just the red channel, and the right eye will see just the blue channel. Because of the disparity between the two images, the brain will again perceive this as if the viewer was observing a real 3D scene.



Teesta suspension bridge-Darjeeling, India





Mark Twain at Pool Table", no date, UCR Museum of Photography

## This is how 3D movies work

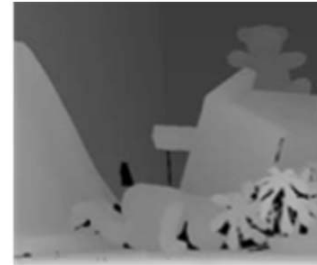


In fact, this disparity-based mechanism is one of the most common ways for displaying 3D movies.

The only difference is that they combine the images using polarization rather than color.

That is, they give you glasses such that each eye is sensitive to a different polarization, and they also use a polarization sensitive screen so they can project a different image to each eye.

*The amount of horizontal movement is  
inversely proportional to ...*



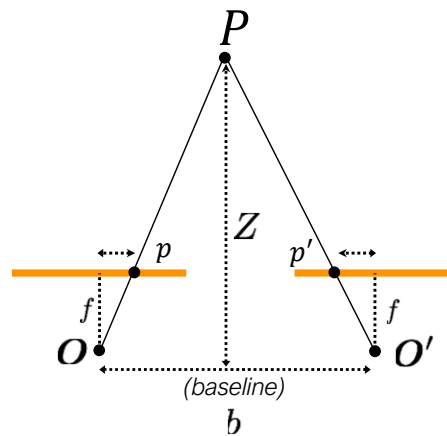
*Disparity image. Brightness in this image  $d(p)$ , encodes the amount of shift between the images at pixel  $p$ .*

*... the distance from the camera.*

*More formally...*

We call the amount of horizontal movement for each pixel the *disparity*. From the observation we discussed earlier, the disparity of a pixel appears to be inversely related to the distance of the object shown in that pixel from the camera; that is, disparity is inversely related to the depth of the object. In the next slides, we will see exactly how to relate disparity to depth, and how to use disparity to compute dense depth maps for a scene, such as what is shown here. We will refer to this procedure as depth from disparity, or stereo.



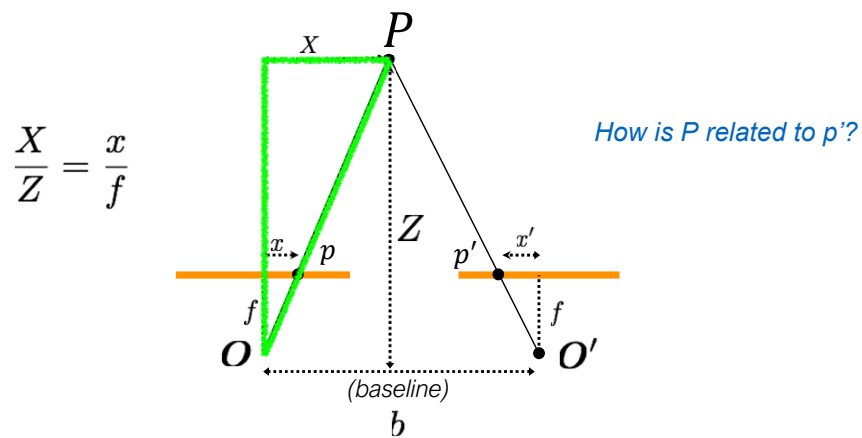


We will describe mathematically the relationship between disparity and object depth. For this, we will use the simple 2D schematic we have here. Some 3D point  $P$  is observed by two cameras, which have the same focal length  $f$ , parallel image planes, but horizontally translated image centers  $O$  and  $O'$ . We end up with the 3D point mapping to image points lower-case  $p$  and  $p'$  on the image planes of the two cameras.

Notice that, in the derivation we will do, we will express image coordinates in its image relative to that image's center.

Note also in this drawing that we can use  $b$  to express the distance between the two camera centers, which is what we described earlier as the baseline.

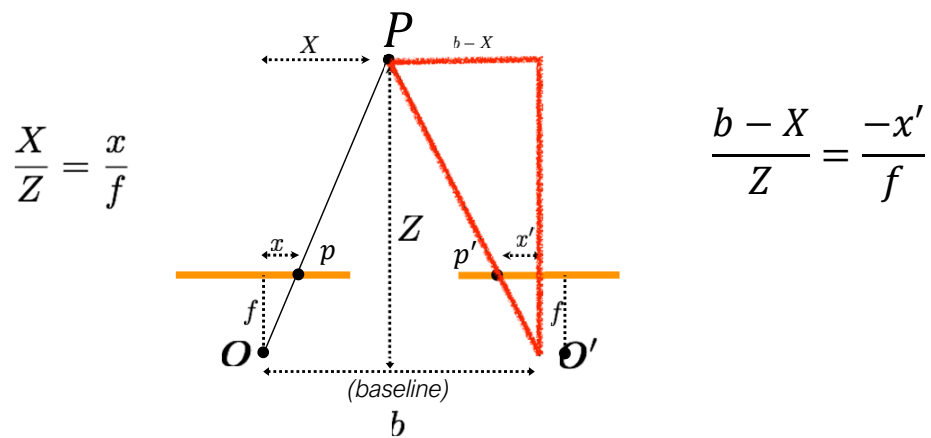
The 3D point  $P$  has depth  $Z$



We denote by  $X$  the first coordinate of the 3D point  $P$ , and by  $x, x'$  the first coordinate of the 2D points  $p, p'$ .

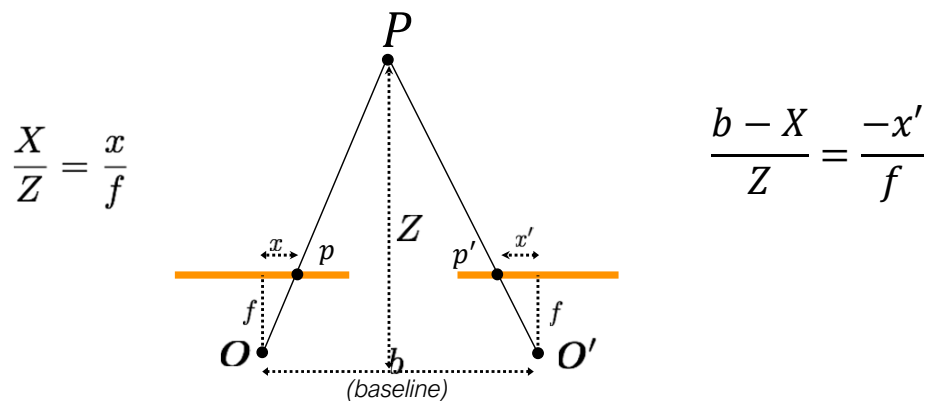
We first want to relate the 3D point upper-case  $X$  to the image coordinate lower-case  $x$  in the left camera.

For this, we can use the similar triangles in green, to form a relationship involving the depth  $Z$  and focal length  $f$ .



We want to derive a similar relationship between the 3D point and the image point on the right camera.

For this, we can use another set of similar triangles, in red, to write down one more expression, now also involving the baseline  $b$ .



$$\frac{X}{Z} = \frac{x}{f}$$

$$\frac{b - X}{Z} = \frac{-x'}{f}$$

### Disparity

$$d = x - x'$$

$$= \frac{bf}{Z}$$

$$Z = \frac{bf}{d}$$

*inversely proportional  
to depth*

By eliminating upper-case  $X$  from the two expressions we derived, we end up with a relationship that relates the difference between the image coordinates, the baseline, the focal length, and the depth.

We call this distance a disparity.

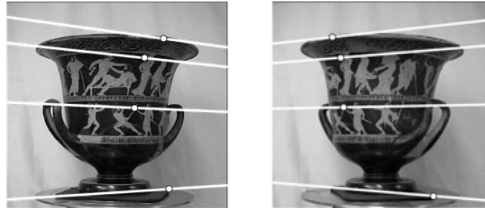
In previous slides we have defined disparity as the shift of a pixel from image 1 to image 2.

Given that the image coordinates are expressed with respect to their respective image center, their difference is exactly what we defined earlier as disparity.

We see, then, that disparity is inversely proportional to depth, with the proportionality factor being equal to the product of the baseline and focal length.

## *When can we use disparity?*

*So can I compute depth from any two images of the same object?*



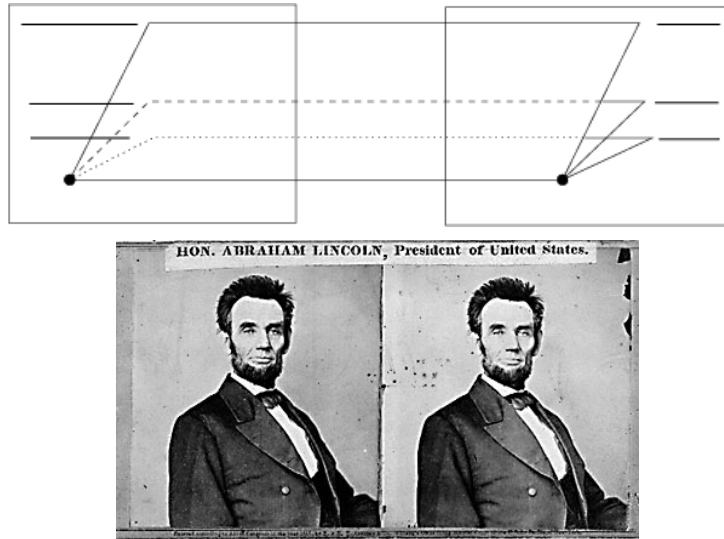
1. Need sufficient baseline  $d = \frac{bf}{Z}$
2. Images need to be 'rectified' first (make epipolar lines horizontal)

It is important to clarify under what conditions we can use this relationship between depth and disparity.

First of all disparity is a function of the baseline and if the baseline is zero we won't be able to recover depth.

Also, the relationship does not apply when we have cameras whose image planes are not parallel, like the two cameras shown here that are verging towards the same scene.

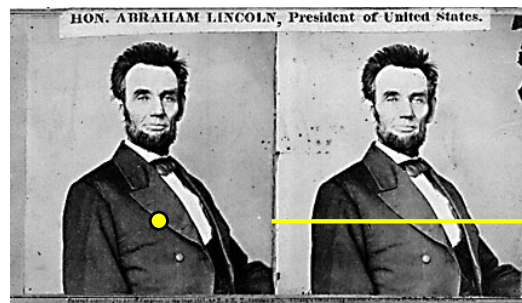
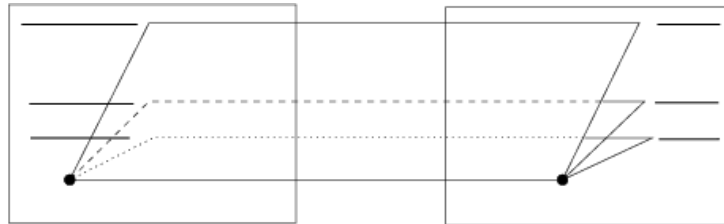
## *When can we use disparity?*



*We can use disparity on images captured only by parallel cameras.*

We can only use disparity on what we will be calling parallel cameras. These are cameras that can be obtained by taking one camera, and translating its center along the x axis of its coordinate system, without any change in the position of the image plane. Parallel cameras have the same focal length, parallel image planes, align coordinate system axes, and a baseline vector that is parallel to the x axis.

## *When can we use disparity?*



*The epipolar line for a point in one image is the corresponding row in the other image.*

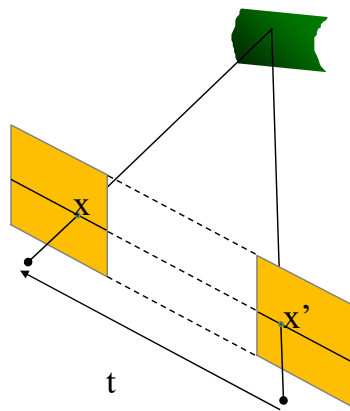
*We can use disparity on images captured only by parallel cameras.*

The use of parallel cameras translates into an important property with respect to the epipolar lines between the two images. Given a point on one image, the epipolar line on the other image will be simply the corresponding row.

# *Stereo rectification*



## *When are two cameras parallel?*

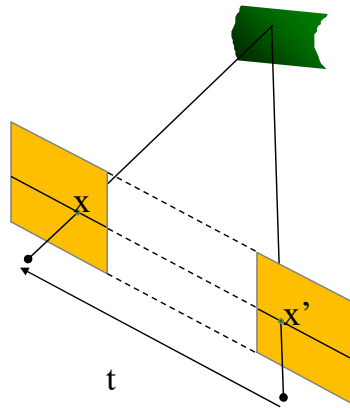


When this relationship holds:

$$R = I \quad t = (T, 0, 0)$$

We can prove this property using the definition of parallel cameras, and what we had derived in earlier lectures about epipolar geometry and the essential matrix. Given the definition of parallel cameras, their extrinsics will be a rotation matrix that is equal to the identity, and a translation vector that is parallel to the x axis.

# When are two cameras parallel?



When this relationship holds:

$$R = I \quad t = (T, 0, 0)$$

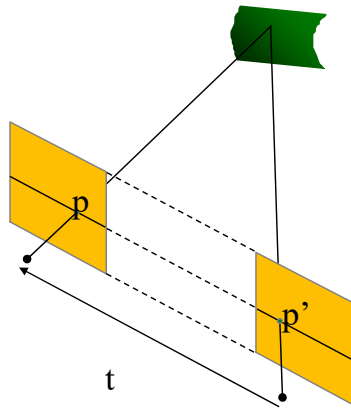
Let's try this out...

$$E = [t_x]R = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -T \\ 0 & T & 0 \end{bmatrix}$$

This always has to hold:  $p'^T E p = 0$

We can plug these extrinsics into the expression we derived earlier for the essential matrix, and use the resulting form of the essential matrix to write out the Longuet-Higgins equation.

# When are two cameras parallel?



When this relationship holds:

$$R = I \quad t = (T, 0, 0)$$

Let's try this out...

$$E = [t_x]R = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -T \\ 0 & T & 0 \end{bmatrix}$$

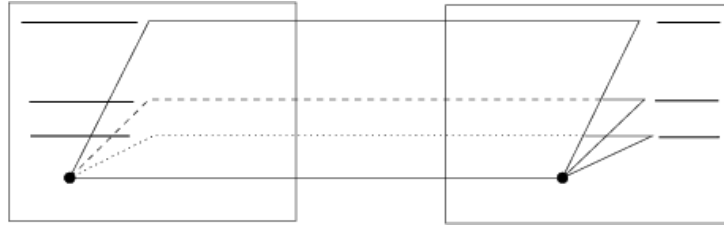
This always has to hold.  $p'^T E p = 0$

$$(x', y', 1) \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -T \\ 0 & T & 0 \end{bmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = 0$$

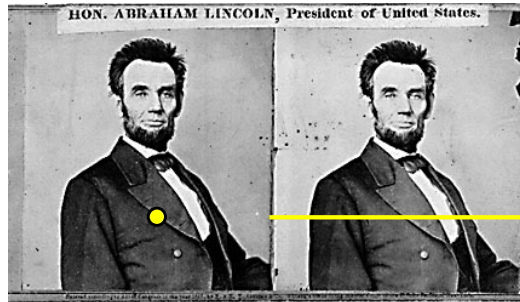
$$\begin{aligned} Ty - Ty' &= 0 \\ y &= y' \\ \text{y coordinate is} \\ \text{always the same!} \end{aligned}$$

Expanding everything out, we see that two points satisfy the Longuet-Higgins equation only if their y coordinates are equal. Therefore, all points that satisfy the epipolar constraint for a point with coordinates  $u, v$  will be points on the row corresponding to y coordinate  $v$ .

# *When can we use disparity?*



*Depth from disparity will only work well for cameras separated by sufficient baseline.*

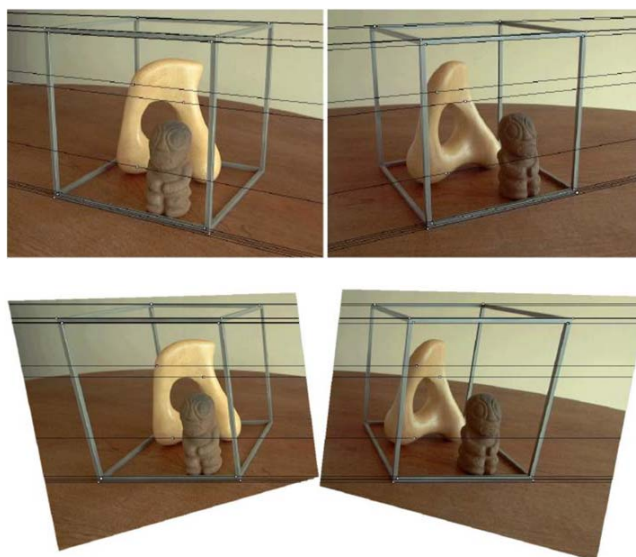


*The epipolar line for a point in one image is the corresponding row in the other image.*

*We can use disparity on images captured only by parallel cameras.*

In practice, in order for disparity to work well, we need to have a camera pair that is separated by a sufficiently large baseline. Unlike the requirement for parallel cameras, this is not a *theoretical* requirement---the theory we derived applies to all baselines. However, when the baseline is very small, because of the *inverse* relationship between disparity and depth, small errors in disparity estimation can result in very large errors in depth estimation.

## Stereo rectification



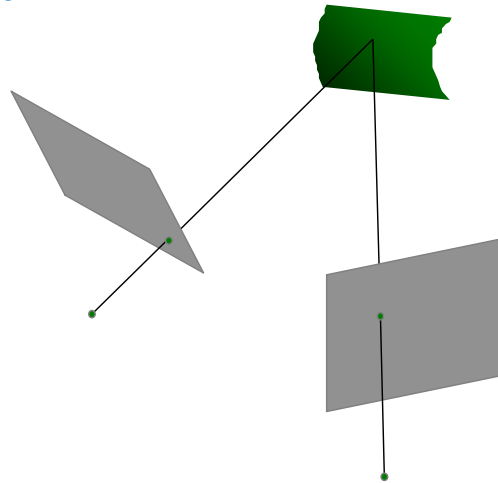
*Warp images captured by non-parallel cameras so that they satisfy conditions for depth from disparity.*

I want to mention a procedure called stereo rectification: This makes it possible to take images captured by two non-parallel cameras, and warp them into images whose corresponding cameras are parallel. In the example shown here, you can see that the original images have epipolar lines that are not horizontal. But the warped images have epipolar lines that are horizontal and at the same y coordinate, as we would expect from parallel cameras.

Stereo rectification is equivalent to applying a 2D homography transformation on the images.

To compute this homography one needs to know the intrinsics and extrinsics of the two cameras that captured the original images.

*What is stereo rectification?*

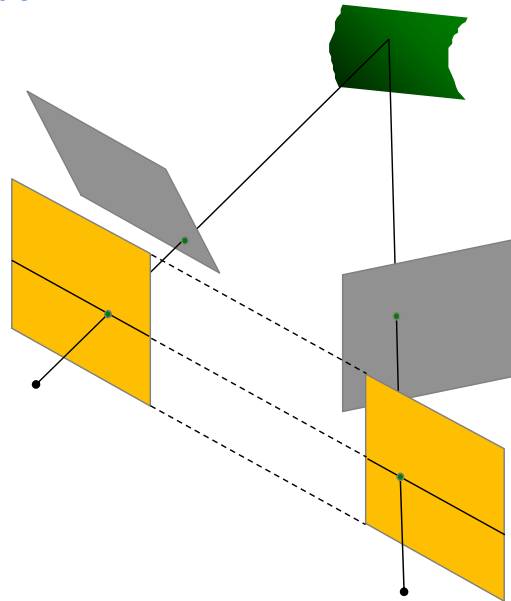


To understand what is stereo rectification consider these two cameras.  
We mark their image planes by two gray planes.

### *What is stereo rectification?*

*Reproject image planes onto a common plane parallel to the line between camera centers*

*How can you do this?*

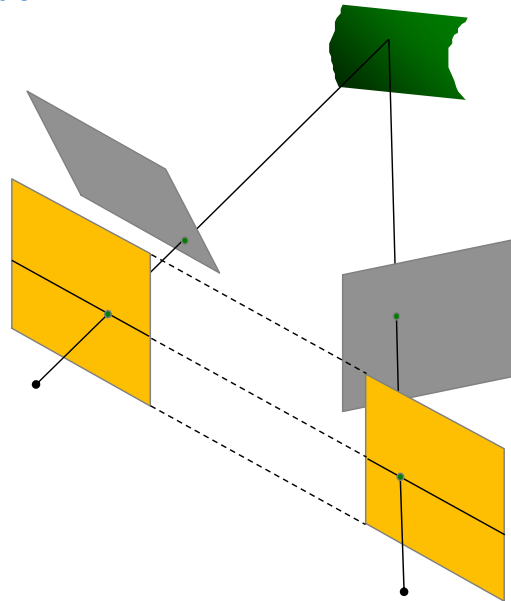


To rectify we want to generate two images taken by two cameras at the same centers but with different image planes marked in yellow. We want to make the new image planes the same plane.

### What is stereo rectification?

*Reproject image planes onto a common plane parallel to the line between camera centers*

*Need two homographies (3x3 transform), one for each input image reprojection*



C. Loop and Z. Zhang. Computing Rectifying Homographies for Stereo Vision. Computer Vision and Pattern Recognition, 1999.

We note that this transformation can be described as a 3x3 homography. The reason is as you have seen in one of the tutorials is that when we have two cameras with the same center, we can map between the images using an homography.



### What is stereo rectification?

Reproject image planes onto a common plane parallel to the line between camera centers

Consider 2 general cameras

$$M_1 = [I | 0]$$

$$M_2 = [R | T]$$

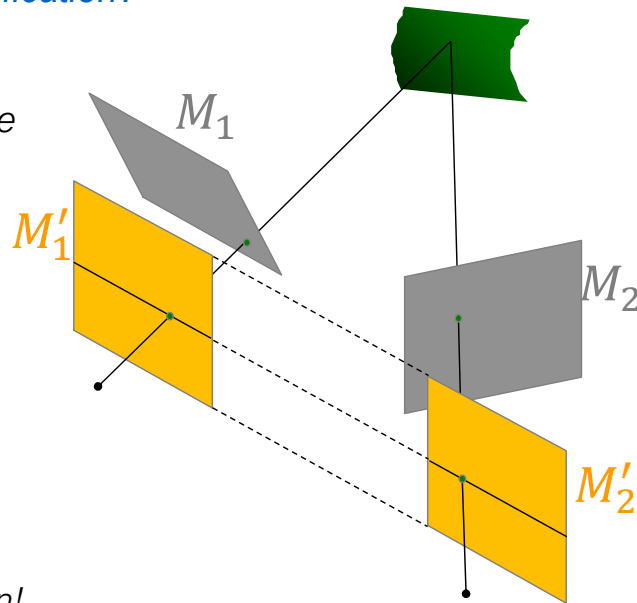
$$p_1 = M_1 P$$

$$p_2 = M_2 P$$

There is no 3x3 mapping:

$$p_2 \neq H p_1$$

Mapping depends on 3D position!



C. Loop and Z. Zhang. Computing Rectifying Homographies for Stereo Vision. Computer Vision and Pattern Recognition, 1999.

Consider two general camera. The first one is  $M_1=[I|0]$  and the 2<sup>nd</sup> one is  $M_2=[R|t]$   
In general there is no 3x3 homography that maps point in image 1 to point in image 2, as this mapping depends on depth, it is not a 2D transformation.

## What is stereo rectification?

Reproject image planes onto a common plane parallel to the line between camera centers

For the specific case of two cameras with a joint center holography mapping does exist!

$$M_1 = [I|0]$$

$$M'_1 = [R'|0]$$

$$p_1 = M_1 P$$

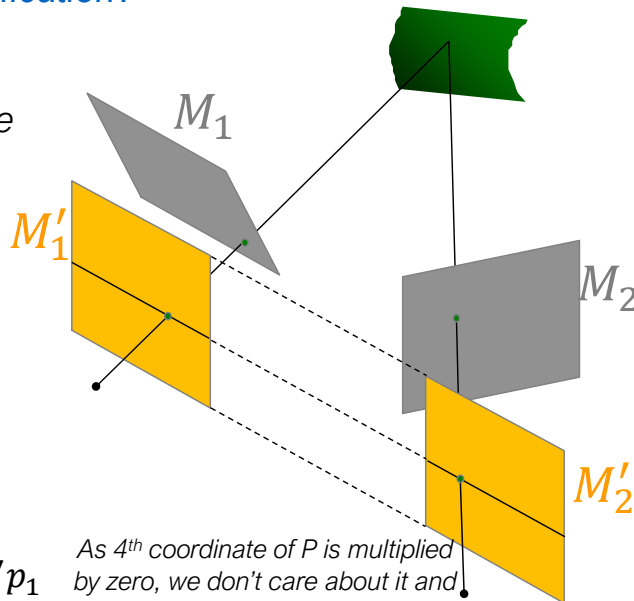
$$p'_1 = M'_1 P$$

Homography mapping  $p'_1 = R' p_1$

simply  $p'_2 = R'_2 p_2$

As 4<sup>th</sup> coordinate of  $P$  is multiplied by zero, we don't care about it and can express mapping as a 3x3 transformation

C. Loop and Z. Zhang. Computing Rectifying Homographies for Stereo Vision. Computer Vision and Pattern Recognition, 1999.



However consider the camera  $M_1$  corresponding to the gray image plane and the camera  $M'_1$  corresponding to the yellow image plane. Both of these cameras have the same center. Therefore if we think of the transformation (in 3D not in 2D) mapping from the coordinate frame of  $M_1$  to the coordinate frame of  $M'_1$ , this transformation is only a rotation with zero translation.

Therefore ignoring for a second the intrinsic parameters we can express  $M'_1 = [R' | 0]$ , that is as a rotation only with no translation.

Therefore we can map the 2D image point  $p_1$  to the 2D image point  $p'_1$  using a 3x3 matrix which happens to be  $R'$ . That is we can write  $p'_1 = R' p_1$ .

Because the two camera matrices  $M_1, M'_1$  have zeros in their 4<sup>th</sup> column, the mapping does not depend on the 4<sup>th</sup> coordinate of  $P$ .

In the same way we can find an homography mapping between the gray image plane of camera 2 to the yellow image plane of camera 2.

If we manage to convert the gray cameras into the yellow cameras we have rectified the images, because we now got two cameras looking at the same direction, with the same image plane and on such cameras epipolar lines are just horizontal lines.

Therefore, stereo rectification is equivalent to finding two homography matrices and applying them on the two images.

There is an algorithm for finding these homographies but we will not get into the details

here.

I will only mention that this algorithm takes as input the camera parameters  $R, t, K$ , and if these are known the rectification homography can be computed with a close-form formula.

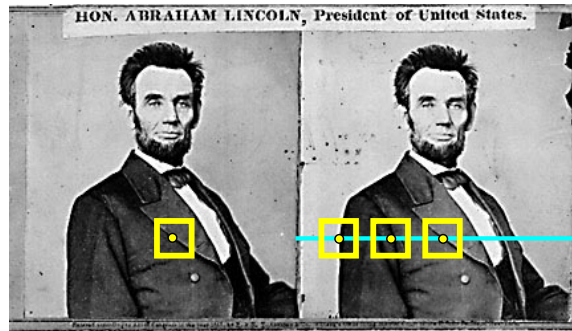
Stereo matching



### *Dense Depth Estimation via Stereo Matching*



Now we have everything we need to write down an algorithm that, given two images of the same scene captured by parallel cameras, can produce a dense depth map, giving us depth values for every pixel. We call this algorithm stereo matching, or simply stereo.



*Stereo matching v.s. triangulation:*  
 1) Finding epipolar lines is trivial, No need to multiply by fundamental matrices  
 2) No need to solve linear system for triangulation

1. Rectify images  
 (make epipolar lines horizontal)
2. For each pixel
  - a. Find epipolar line  $e_z$
  - b. Scan line for best match
  - c. Compute depth from disparity

*How would you do this?*

$$Z = \frac{bf}{d}$$

Here is how this works. First, if our two images are not captured by parallel cameras, we need to run the stereo rectification algorithm as preprocessing.

Once our images are rectified, we now run a double for loop over each pixel on the left image. For each pixel, we perform three steps:

First, we find the corresponding epipolar line on the right image. Finding the epipolar line is trivial here: It is just the row matching the y coordinate of the pixel in the left image we are processing.

Second, we scan the epipolar line, for the pixel whose appearance best matches the pixel in the left image. We will see how to do this in a few minutes.

Once we have found the best match, we can compute disparity as the difference between the locations of the two matching pixels. Then, by inverting this disparity, we have the depth of that pixel. Compared to the triangulation-based procedure, the stereo matching procedure is simpler in two ways: First, we do not need to use the fundamental matrix to find the epipolar lines, and instead can simply look at the same row in the other image. Second, once we have found two matching

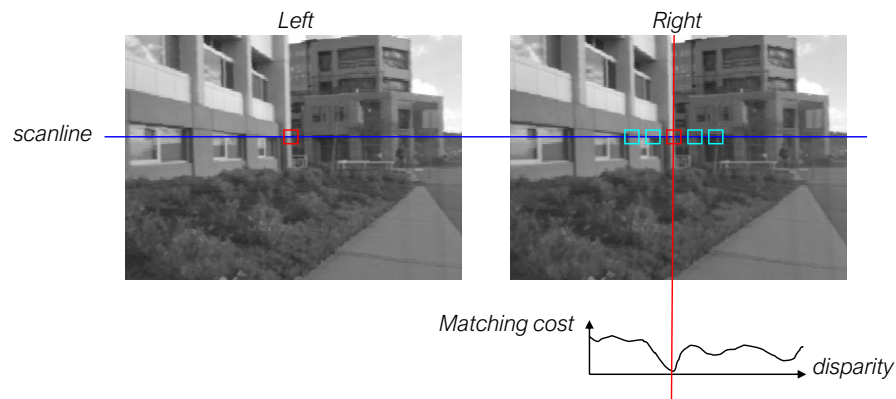
pixels, we do not need to solve a linear system to compute the 3D point. Instead, we get depth simply by doing a scalar inversion. These two differences make it very simple to use stereo matching to compute depth for all pixels in an image.

So how do we match pixels in the two images?

For this, we can potentially use the feature descriptors we discussed in the Feature Detection and Matching module: We can compute a feature descriptor for every pixel on the epipolar line and compare it to the feature descriptor of the pixel in the left image. However, ultimately this would require computing descriptors for each pixel in both images, which can be computationally very expensive.

Instead, we can do matching using a simpler template matching procedure. By template matching we mean loss functions that compute, on each pixel of an image, how similar the patch surrounding that pixel is to some target template.

# Stereo Block Matching



- Slide a window along the epipolar line and compare contents of that window with the reference window in the left image
- Matching cost: SSD or normalized correlation

One commonly used loss function is the sum of squared differences, where we go over each pixel in the candidate patch, compare it to each pixel in the template, and sum the squared error.

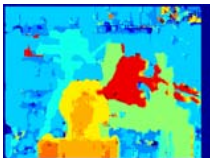
This approach can fail if the template and the candidate patches are similar in structure, but differ by some constant intensity scaling. Therefore, another commonly used loss function is the normalized cross-correlation, or NCC.

For every location along the epipolar line, we can plot either the SSD or the NCC score, then select a pixel that has the best score.

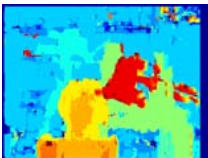
Here is an example using SSD, where the matching pixel indeed produces the smallest SSD error.



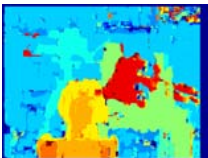
| Similarity Measure                 | Formula                                                                                                                               |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Sum of Absolute Differences (SAD)  | $\sum_{(i,j) \in W}  I_1(i,j) - I_2(x+i,y+j) $                                                                                        |
| Sum of Squared Differences (SSD)   | $\sum_{(i,j) \in W} (I_1(i,j) - I_2(x+i,y+j))^2$                                                                                      |
| Zero-mean SAD                      | $\sum_{(i,j) \in W}  I_1(i,j) - \bar{I}_1(i,j) - I_2(x+i,y+j) + \bar{I}_2(x+i,y+j) $                                                  |
| Locally scaled SAD                 | $\sum_{(i,j) \in W}  I_1(i,j) - \frac{\bar{I}_1(i,j)}{\bar{I}_2(x+i,y+j)} I_2(x+i,y+j) $                                              |
| Normalized Cross Correlation (NCC) | $\frac{\sum_{(i,j) \in W} I_1(i,j) \cdot I_2(x+i,y+j)}{\sqrt{\sum_{(i,j) \in W} I_1^2(i,j) \cdot \sum_{(i,j) \in W} I_2^2(x+i,y+j)}}$ |



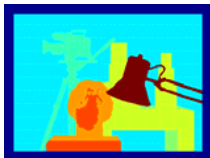
SAD



SSD



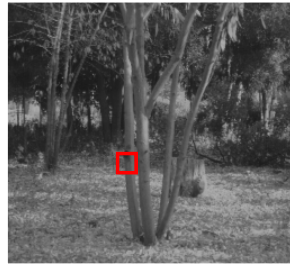
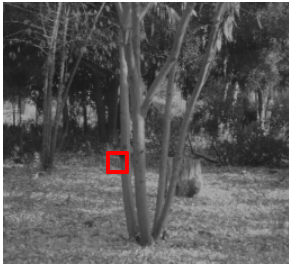
NCC



Ground truth

Here is a list of some commonly used metrics for comparing patches.

## Effect of window size



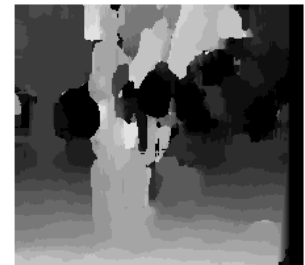
*This patch contains pixels at two different depth, cannot determine unique disparity*



$W = 3$

### **Smaller window**

- + More detail
- More noise



$W = 20$

### **Larger window**

- + Smoother disparity maps
- Less detail
- Fails near boundaries

Both of these matching procedures require selecting a window size around each pixel on the epipolar line. The window size will determine the patch we use to determine the matching loss. The window size is an important hyperparameter that can significantly affect the performance of stereo matching. For example, using a small window gives us a lot more detail in the resulting depth map, but also results in a lot more noise. Conversely, using a large window results in less noisy depth maps, but also in depth maps that have less detail and overblur along object boundaries.



No match in right  
image due to  
occlusion

نه در تصویر راست  
همخوانی نیست به  
سبب پوشش

Is that a good  
match? Pixels on  
the house match,  
background pixels  
inside the window  
do not match

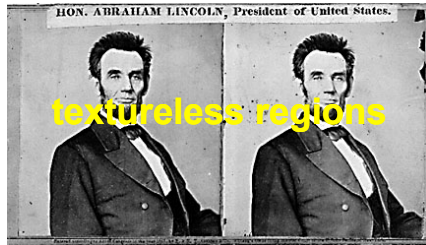
Let look at this stereo pair to see what are all the problems we get next to image boundaries.

Let's start with this orange window. It is easy to define what is the corresponding window in image 2.

Now let's look at this red patch. We do not have a match in image 2 as this part of the scene is occluded.

Now let's look at this purple window. We would like to say that it matches to this window. But is it really a good match? The pixels inside this window belonging to the house match. But the background (white) pixels of the window actually don't match. If we force our algorithm to make a decision it will select some disparity value here, but next to occlusion boundaries disparity estimation is often wrong.

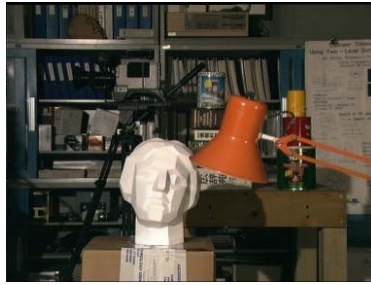
### *When will stereo block matching fail?*



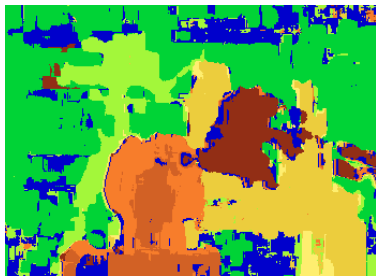
سوال  
نه  
نه  
نه  
نه

Lastly, it is important to be aware of three cases that can cause the stereo matching algorithm to fail. One is flat regions in the image, where there are no features we can use to reliably find stereo matches. A second is regions with repeated patterns, that can fool the stereo matching procedure into matching incorrect, but similarly looking, pixels. The third case are mirror-like objects, which can drastically change appearance between two views, resulting in a lot of incorrect matches.

Improving stereo matching



*Block matching*

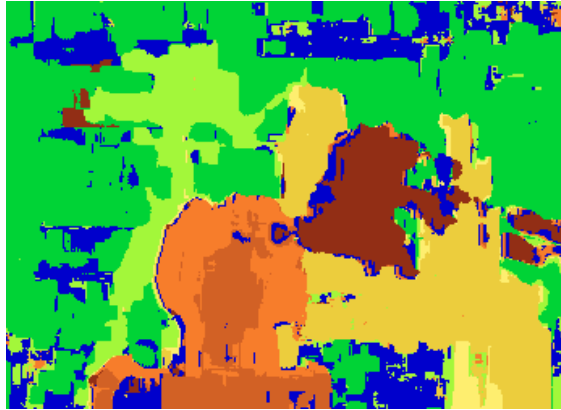


*Ground truth*



*What are some problems with the result?*

In the previous slides, we described stereo matching as a computationally efficient algorithm for computing dense depth maps from pairs of images of the same scene. While being computationally efficient, the stereo matching algorithm can also often produce erroneous depth estimates, as shown in this example. The main reason for these errors is inaccurate pixel matching during the stereo matching procedure.



*How can we improve depth estimation?*

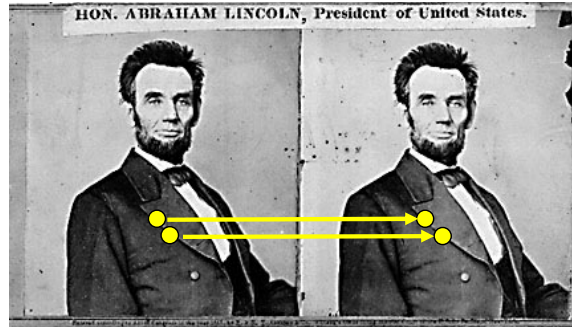
*Too many discontinuities.  
We expect disparity values to change slowly.*

*Let's make an assumption:  
**depth should change smoothly***

We can reduce the amount of errors by making an assumption about the depth maps we expect to have: We expect depth values to change smoothly along most pixels in the image, except along object boundaries.

Stereo matching as ...

# Energy Minimization



What defines a good stereo correspondence?

- 1. Match quality**

- Want each pixel to find a good match in the other image

- 2. Smoothness**

- If two pixels are adjacent, they should (usually) move about the same amount

With this assumption in mind, we can frame stereo matching as a problem that tries to produce stereo correspondences satisfying two, possibly competing, criteria: First, we want the matched pixels to have similar appearance. For example, the patches around the two pixels should have a low SSD loss. Second, we want the stereo correspondences to correspond to a smooth disparity map: By this we mean that, if two pixels in the left image are adjacent, their matched pixels in the right image should also be close to each other.



*energy function  
(for one pixel)*

$$E(d) = \underbrace{E_d(d)}_{\text{data term}} + \lambda \underbrace{E_s(d)}_{\text{smoothness term}}$$

*data term*

*smoothness term*

*Want each pixel to find a good match  
in the other image  
(block matching result)*

*Adjacent pixels should (usually)  
move about the same amount  
(smoothness function)*

We can then determine the disparity  $d$  of each pixel as the solution to an energy minimization problem, whose loss function combines two terms: The first is a term dictating that the disparity value assigned to the pixel should result in closely-matching image patches. The second term dictates that the disparity we assign to this pixel should be similar to the disparity values of its neighbors.

$$E(d) = E_d(d) + \lambda E_s(d)$$

$$E_d(d) = \sum_{(x,y) \in I} C(x, y, d(x, y))$$

*data term*

*SSD distance between windows  
centered at  $I(x, y)$  and  $J(x + d(x, y), y)$*

For the data term, we can use, for example, the SSD loss. For each disparity value, we look at the corresponding pixel on the right image, and compute the SSD score between the patch around that pixel, and the patch around the pixel in the left image we are trying to match.

$$E(d) = E_d(d) + \lambda E_s(d)$$

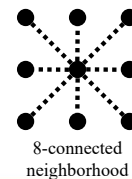
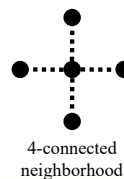
$$E_d(d) = \sum_{(x,y) \in I} C(x, y, d(x, y))$$

SSD distance between windows  
centered at  $I(x, y)$  and  $J(x + d(x, y), y)$

$$E_s(d) = \sum_{(p,q) \in \mathcal{E}} V(d_p, d_q)$$

smoothness term

$\mathcal{E}$  : set of neighboring pixels



For the smoothness term, we can first define some neighborhood around the pixel we are looking to match. This can be either the 4-connected or the 8-connected neighborhoods shown here.

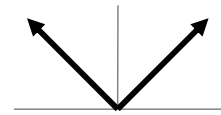
Then, the smoothness loss can be the sum of pairwise differences between the disparities assigned to the pixel we are trying to match, and its neighbors.

smoothness term

$$E_s(d) = \sum_{(p,q) \in \mathcal{E}} V(d_p, d_q)$$

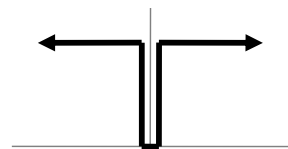
$$V(d_p, d_q) = \text{aff}(I_p, I_q) |d_p - d_q|$$

$L_1$  distance



$$V(d_p, d_q) = \text{aff}(I_p, I_q) \begin{cases} 0 & \text{if } d_p = d_q \\ 1 & \text{if } d_p \neq d_q \end{cases}$$

"Potts model"



$$\text{aff}(I_p, I_q) = e^{\frac{-|I_p - I_q|^2}{\sigma}}$$

Penalty for discontinuities is lower if edge in the image exists

Pairwise differences can be measured using either the absolute difference, or even just a binary loss.

It is also common to use an affinity which depends on image gradient. This loss states that if we assign neighboring pixels different depth values, we pay more if their colors are the same and we pay a lower penalty if their colors are different.

The idea is that if two neighboring pixels come from objects at different depths, these are probably two different objects and they probably also have different colors.

Said differently depth edges and color edges are located in the same place in the image.

## *Energy minimization*

$$E(d) = E_d(d) + \lambda E_s(d)$$

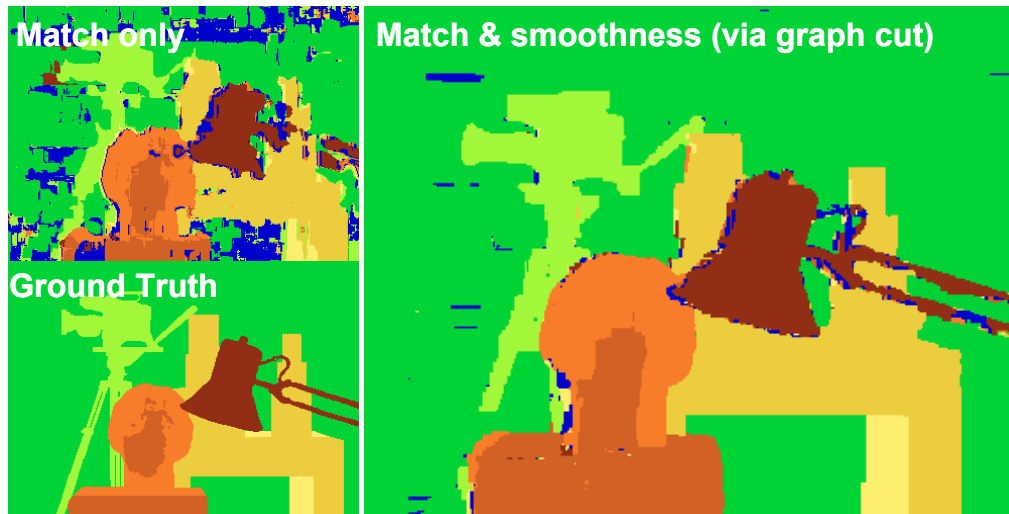
*Can minimize this using the graph cut  
framework presented for segmentation*

We can minimize this type of cost function using various approaches.

One algorithm that we have met earlier in the course in the context of segmentation, is graph cuts.

Graph cuts were widely used to minimize the stereo matching cost.

Clearly, nowadays stereo matching is solved with neural networks.



Y. Boykov, O. Veksler, and R. Zabih, [Fast Approximate Energy Minimization via Graph Cuts](#), PAMI 2001

Going back to the example we used earlier, we can see that using the optimization approach can help deal with the issue of incorrect stereo matches, and thus produce significantly improved depth maps.

## *Stereo with energy minimization*

*We can now use very small patches usually 3x3 or even 1x1*

*Advantages of small patches: less problems at occlusion boundaries.*

*Disadvantages of small patches: local matching is noisy.*

*But once we have smoothness in our cost, noisy local matchings are smoothed.*

*Thus, with energy minimization we can get both clean results and crisp occlusion boundaries*

## *Stereo v.s. structure from motion*

*Structure from motion: needs matching features, usually only a sparse point cloud for which we can find unique features (corners etc).*

*Stereo: dense match for every point in the image. Need to get camera geometry as input (for rectification).*

*Approach 1: place two cameras on a fixed rig and they do not move (car dashboard, two cameras on a cellphone, etc) and they are really parallel.*

*Approach 2: For handheld cameras, use structure from motion as a first step before stereo. Detect a sparse set of features and calibrate cameras, and then use camera parameters to do stereo matching.*



Structured light

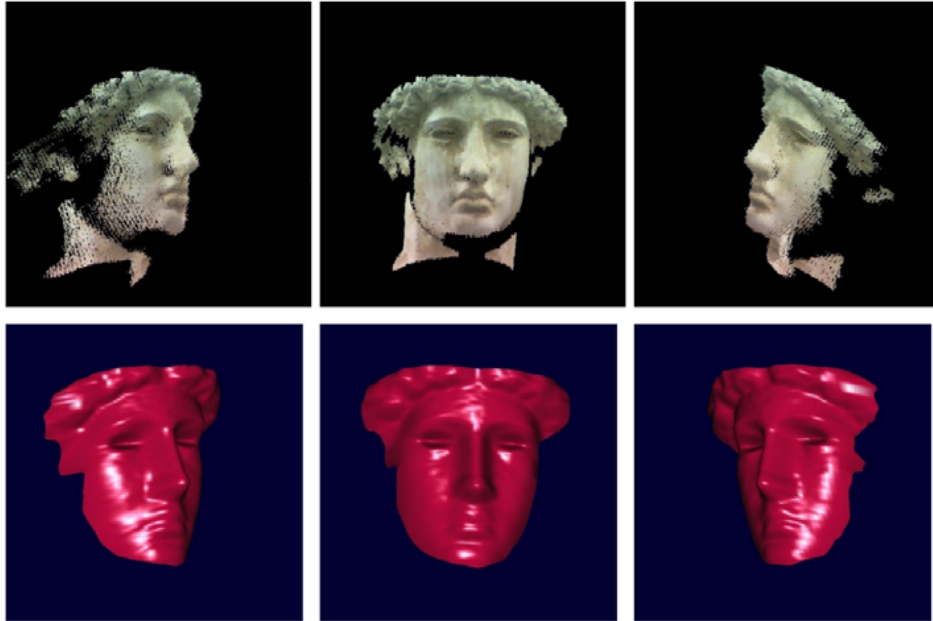
Use controlled (“structured”) light to make correspondences easier

Disparity between laser points on the same scanline in the images determines the 3-D coordinates of the laser point on object

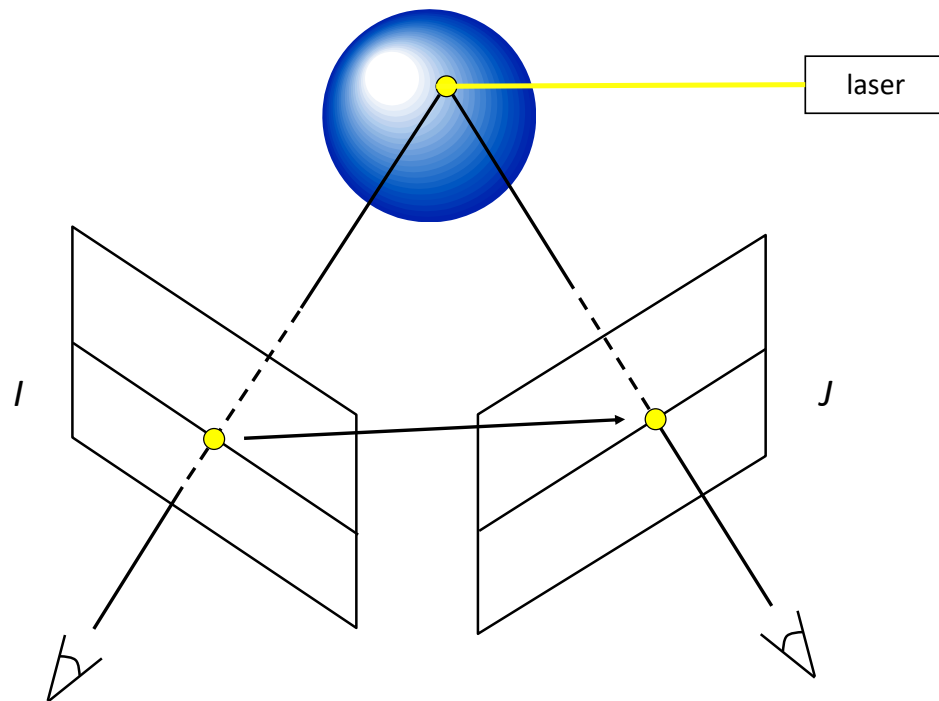


Throughout this lecture, we have focus on so-called *passive* 3D sensing techniques: This means that we use images we capture from a scene under ambient light, for example due to the sun or room light sources, and we cannot change how we illuminate the scene. In the coming slides, we will quickly go over one of the most commonly used *active* 3D sensing technique, known as structured light. The high-level idea of structure light is that, by using a laser source to illuminate specific parts of the scene we are reconstructing, we can make stereo matching a lot easier, and therefore produce very high-detailed depth reconstructions.

Use controlled (“structured”) light to make correspondences easier



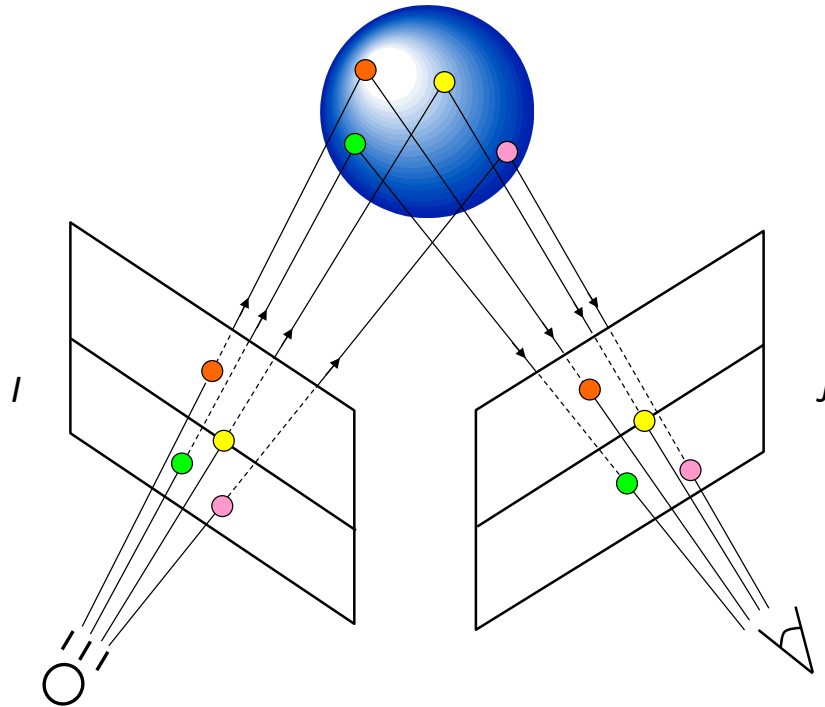
## Structured light and two cameras



Here is how this works. Given two cameras observing the scene, we can use an external light source to illuminate different points in the scene. The illuminated point with light up in the images captured by the two cameras, and we can detect it using a simple thresholding operation. We no longer need to use SSD, NCC, feature detectors, or anything else to do stereo matching. Once we have the two points, we can use either triangulation or disparity to compute depth. We scan the laser beam over the target object and compute the depth at every point.

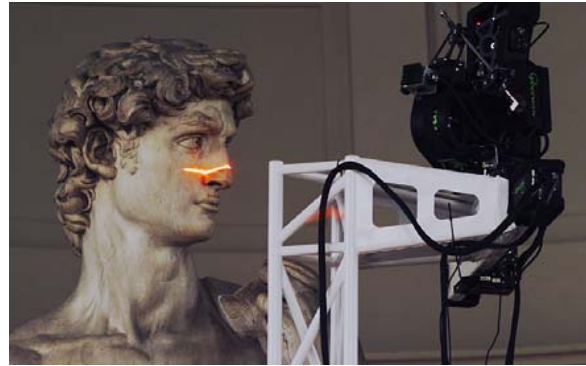
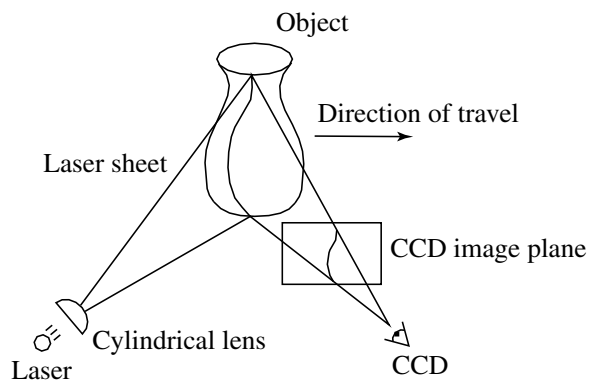
## Structured light and one camera

Projector acts like  
“reverse” camera



Alternatively, we can replace one of the two cameras with a projector. A projector is mathematically exactly equivalent to a pinhole camera, except that instead of receiving and measuring light rays, it is sending light rays out. We can then illuminate a pixel on the projector, observe from the camera what pixel on the image plane lights up, and then use the two pixel locations for 3D reconstruction.

## Example: Laser scanner



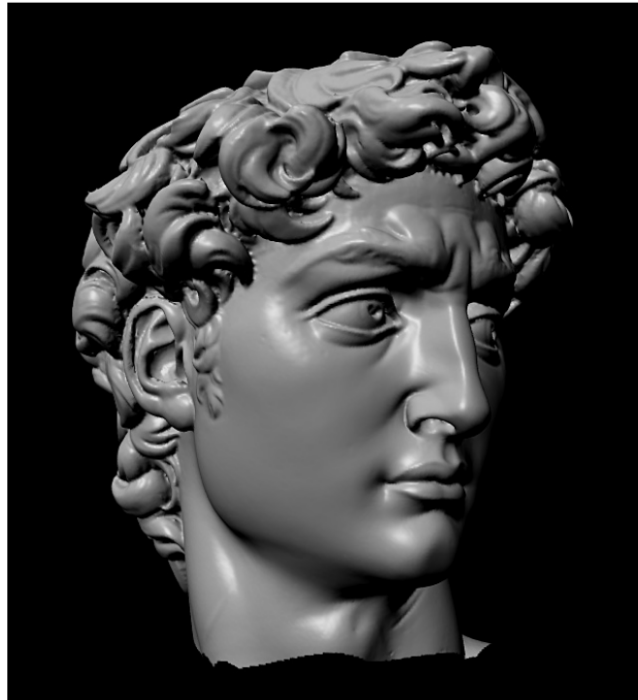
Digital Michelangelo Project  
<http://graphics.stanford.edu/projects/mich/>

Using this laser scanning procedure, we can obtain incredibly high resolution 3D scans of even very detailed scenes. I will show here some examples of the digital michelangelo project from stanford, where structured light scanning was used to digitize various works of art by Michelangelo, including the famous david statue.



*The Digital Michelangelo Project, Levoy et al.*

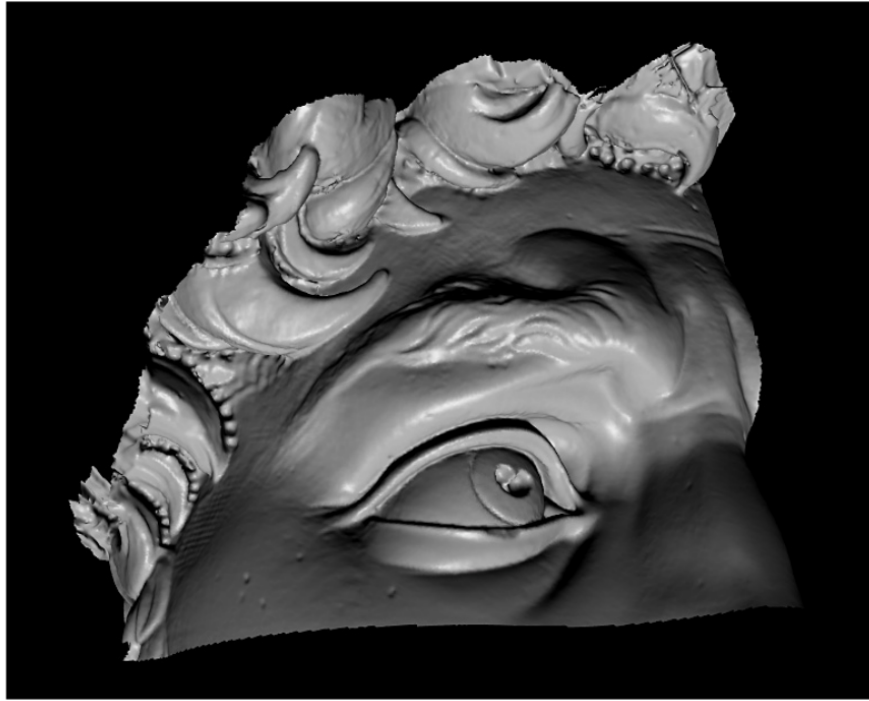
Here is an image of david, in case you do not know what statue I am talking about.



*The Digital Michelangelo Project, Levoy et al.*

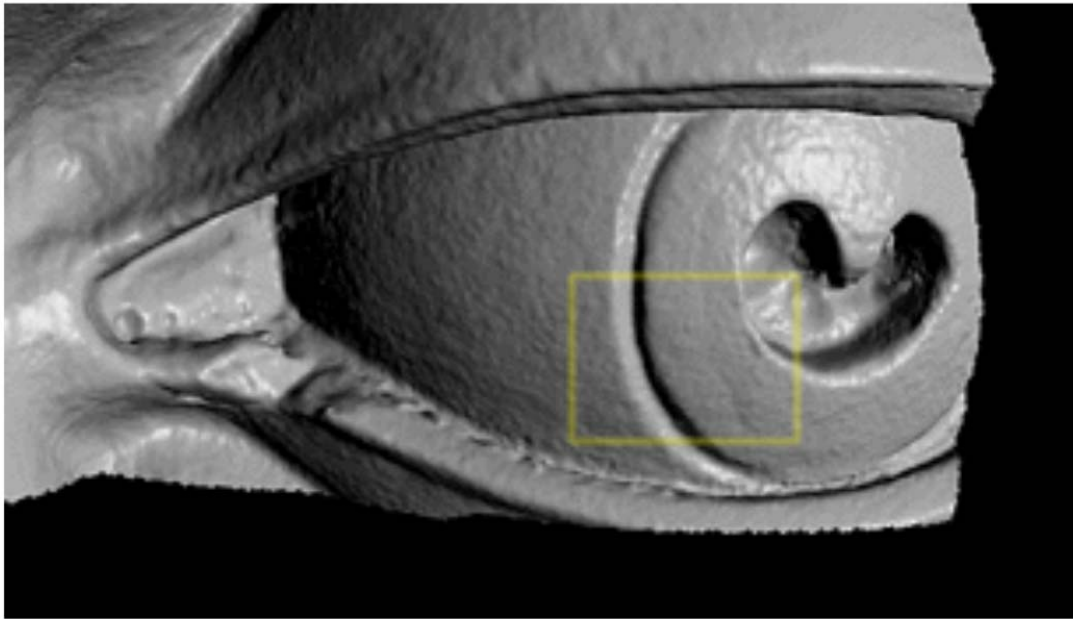
And here is a visualization of the scanned 3D surface of the statue's head.





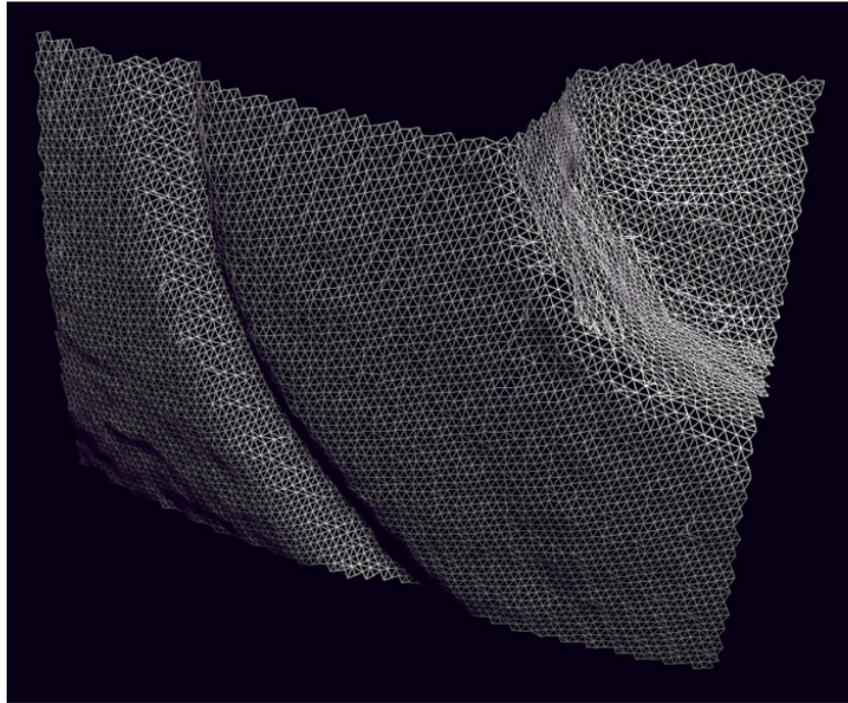
*The Digital Michelangelo Project, Levoy et al.*

We can zoom in to see some of the incredible detail that is captured here, like the details on the eyebrow and the hair.



*The Digital Michelangelo Project, Levoy et al.*

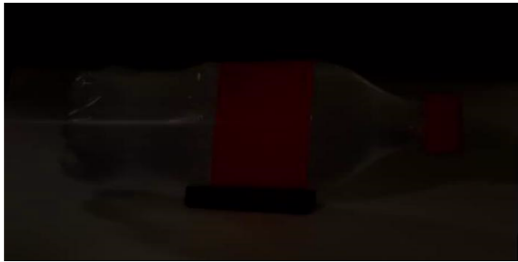
We can zoom even further to see that the 3D scanning procedure even manages to correctly reconstruct details such as the charged eye pupil at the statute. Of course, all of this detail comes at the cost that we need to be able to use a light source to illuminate our scene, something that may not always be practical. But, if it is, it is worth having structured light in mind, as an active way for performing 3D sensing at very high resolutions.



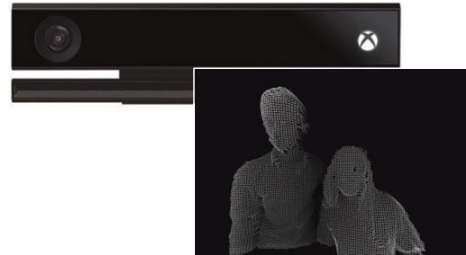
*The Digital Michelangelo Project, Levoy et al.*

# Computational Photography

Learn about structured light and other  
(Come to my class: spring 2022)



cameras that take video at the speed of light



cameras that measure depth in real time



cameras that see around corners



cameras that capture  
entire focal stacks

# How do we stitch images from different viewpoints?



Idea 1: Translate one image relative to another.

## How do we stitch images from different viewpoints?



Idea 1: Translate one image relative to another.

left on top



right on top

Translation-only stitching is not enough to mosaic these images.

How do we stitch images from different viewpoints?



What else can we try?

How do we stitch images from different viewpoints?



Use image homographies.





## We can use homographies when...

1. ... the scene is planar; or



2. ... the scene is very far or has small (relative) depth variation  
→ scene is approximately planar



## We can use homographies when...

3. ... the scene is captured under camera rotation only (no translation or pose change)



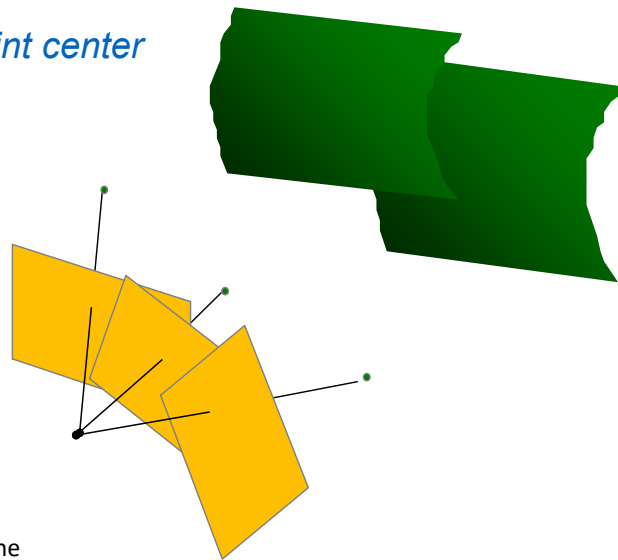
More on why this is the case in a later lecture.

*Homography can map images with a joint center*

$$\begin{aligned}M_1 &= [I|0] \\M_2 &= [R \ |0] \\p &= M_1 P \\p' &= M_2 P\end{aligned}$$

Homography mapping  $p' = R \ p$

We don't care about the  
4th coordinate of P



On the other hand: joint camera centers implies that 3D reconstruction is not possible!

C. Loop and Z. Zhang. Computing Rectifying Homographies for Stereo Vision. Computer Vision and Pattern Recognition, 1999.

# References

Basic reading:

- Szeliski textbook, Section 8.1 (not 8.1.1-8.1.3), Chapter 11, Section 12.2.
- Hartley and Zisserman, Section 11.12.